

```
In [41]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [42]: df = pd.read_excel("Data Cleaning 12.08.26.xlsx")
df
```

Out[42]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini
0	109 Psc b	Confirmed	5.743	0.28900	1.01100	6.38300	
1	112 Psc b	Confirmed	NaN	0.00500	0.00400	0.03300	
2	112 Psc c	Confirmed	9.866	1.78100	3.19000	NaN	
3	11 UMi b	Confirmed	NaN	1.10000	1.10000	11.08730	
4	14 And Ab	Confirmed	NaN	0.23000	0.23000	4.68400	
...	...	...	...	...	...	...	...
5981	YBP 401 b	Confirmed	NaN	0.05000	0.05000	0.46000	
5982	YSES 2 b	Confirmed	6.300	0.90000	1.60000	NaN	
5983	YZ Cet b	Confirmed	NaN	0.00028	0.00028	0.00220	
5984	YZ Cet c	Confirmed	NaN	0.00035	0.00035	0.00359	
5985	YZ Cet d	Confirmed	NaN	0.00038	0.00038	0.00343	

5986 rows × 10 columns



```
In [43]: df.head()
```

Out[43]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_err
0	109 Psc b	Confirmed	5.743	0.289	1.011	6.3830	
1	112 Psc b	Confirmed	NaN	0.005	0.004	0.0330	
2	112 Psc c	Confirmed	9.866	1.781	3.190	NaN	
3	11 UMi b	Confirmed	NaN	1.100	1.100	11.0873	
4	14 And Ab	Confirmed	NaN	0.230	0.230	4.6840	

In [44]: `df.tail()`

Out[44]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_err
5981	YBP 401 b	Confirmed	NaN	0.05000	0.05000	0.46000	
5982	YSES 2 b	Confirmed	6.3	0.90000	1.60000	NaN	
5983	YZ Cet b	Confirmed	NaN	0.00028	0.00028	0.00220	
5984	YZ Cet c	Confirmed	NaN	0.00035	0.00035	0.00359	
5985	YZ Cet d	Confirmed	NaN	0.00038	0.00038	0.00343	

In [45]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5986 entries, 0 to 5985
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   name                   5986 non-null   object
1   planet_status          5986 non-null   object
2   mass                   2079 non-null   float64
3   mass_error_min         2817 non-null   float64
4   mass_error_max         2817 non-null   float64
5   mass_sini              1202 non-null   float64
6   mass_sini_error_min    1039 non-null   float64
7   mass_sini_error_max    1039 non-null   float64
8   radius                 4568 non-null   float64
9   radius_error_min       4093 non-null   float64
dtypes: float64(8), object(2)
memory usage: 467.8+ KB
```

```
In [46]: df.isnull()
```

Out[46]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_sini
0	False	False	False	False	False	False	
1	False	False	True	False	False	False	
2	False	False	False	False	False	True	
3	False	False	True	False	False	False	
4	False	False	True	False	False	False	
...	...	...	...	...	...	...	...
5981	False	False	True	False	False	False	
5982	False	False	False	False	False	True	
5983	False	False	True	False	False	False	
5984	False	False	True	False	False	False	
5985	False	False	True	False	False	False	

5986 rows × 10 columns



```
In [47]: df.isnull().sum()
```

Out[47]:

name	0
planet_status	0
mass	3907
mass_error_min	3169
mass_error_max	3169
mass_sini	4784
mass_sini_error_min	4947
mass_sini_error_max	4947
radius	1418
radius_error_min	1893

dtype: int64

```
In [48]: df.shape
```

Out[48]: (5986, 10)

```
In [49]: df.duplicated()
```

```
Out[49]: 0      False
         1      False
         2      False
         3      False
         4      False
         ...
        5981    False
        5982    False
        5983    False
        5984    False
        5985    False
        Length: 5986, dtype: bool
```

```
In [50]: df.dtypes
```

```
Out[50]: name                object
         planet_status        object
         mass                 float64
         mass_error_min       float64
         mass_error_max       float64
         mass_sini            float64
         mass_sini_error_min   float64
         mass_sini_error_max   float64
         radius               float64
         radius_error_min      float64
         dtype: object
```

```
In [51]: df.columns
```

```
Out[51]: Index(['name', 'planet_status', 'mass', 'mass_error_min', 'mass_error_max',
               'mass_sini', 'mass_sini_error_min', 'mass_sini_error_max', 'radius',
               'radius_error_min'],
              dtype='object')
```

```
In [52]: df.describe()
```

```
Out[52]:
```

	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min
count	2079.000000	2817.000000	2817.0000	1202.000000	1039.000000
mean	1.492538	0.324366	inf	1.983529	0.214687
std	2.602286	1.351929	NaN	3.199717	0.636474
min	0.000190	0.000000	-17.8000	0.000470	0.000000
25%	0.029655	0.006290	0.0057	0.045000	0.005940
50%	0.370000	0.041000	0.0400	0.731500	0.049300
75%	1.466500	0.170000	0.1700	2.400000	0.170000
max	12.934000	53.500000	inf	50.000000	13.000000

```
In [53]: df.dropna(inplace = True)
         df
```

Out[53]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_si
29	51 Peg b	Confirmed	0.470	0.070	0.030	0.4600	
79	BD+20 594 b	Confirmed	0.070	0.030	0.030	0.0513	
90	beta Pic c	Confirmed	8.890	0.750	0.750	8.5000	
120	CoRoT-29 b	Confirmed	0.850	0.200	0.200	0.8500	
370	HAT-P-50 b	Confirmed	1.360	0.075	0.075	1.3500	
...	...	...	...	...	...	...	
5933	WASP-83 b	Confirmed	0.300	0.030	0.030	0.3000	
5936	WASP-85 Ab	Confirmed	1.265	0.065	0.065	1.2650	
5937	WASP-87 Ab	Confirmed	2.210	0.160	0.160	2.1800	
5939	WASP-89 b	Confirmed	5.900	0.400	0.400	5.9000	
5974	XO-4 b	Confirmed	1.616	0.100	0.101	1.6150	

112 rows × 10 columns



In [54]:

```
df.fillna(df.median(numeric_only=True), inplace=True)
df
```

Out[54]:

	name	planet_status	mass	mass_error_min	mass_error_max	mass_sini	mass_si
29	51 Peg b	Confirmed	0.470	0.070	0.030	0.4600	
79	BD+20 594 b	Confirmed	0.070	0.030	0.030	0.0513	
90	beta Pic c	Confirmed	8.890	0.750	0.750	8.5000	
120	CoRoT-29 b	Confirmed	0.850	0.200	0.200	0.8500	
370	HAT-P-50 b	Confirmed	1.360	0.075	0.075	1.3500	
...	...	...	...	...	...	...	...
5933	WASP-83 b	Confirmed	0.300	0.030	0.030	0.3000	
5936	WASP-85 Ab	Confirmed	1.265	0.065	0.065	1.2650	
5937	WASP-87 Ab	Confirmed	2.210	0.160	0.160	2.1800	
5939	WASP-89 b	Confirmed	5.900	0.400	0.400	5.9000	
5974	XO-4 b	Confirmed	1.616	0.100	0.101	1.6150	

112 rows × 10 columns



```
In [55]: df["name"] = df["name"].astype(str).str.strip()
```

```
In [56]: df["planet_status"] = (  
    df["planet_status"]  
    .astype(str)  
    .str.strip()  
    .str.title()  
)  
print(df["planet_status"].value_counts())
```

```
planet_status  
Confirmed    112  
Name: count, dtype: int64
```

```
In [57]: numeric_columns = df.select_dtypes(include="number").columns  
df[numeric_columns].lt(0).sum()
```

```
Out[57]: mass                0
mass_error_min            0
mass_error_max            0
mass_sini                 0
mass_sini_error_min       0
mass_sini_error_max       0
radius                   0
radius_error_min          0
dtype: int64
```

```
In [58]: df[numeric_columns] = df[numeric_columns].mask(df[numeric_columns] < 0)
```

```
In [59]: df[numeric_columns].lt(0).sum()
```

```
Out[59]: mass                0
mass_error_min            0
mass_error_max            0
mass_sini                 0
mass_sini_error_min       0
mass_sini_error_max       0
radius                   0
radius_error_min          0
dtype: int64
```

```
In [60]: for col in numeric_columns:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower_limit) |
        (df[col] > upper_limit)
    ]

    print(col, "Outliers:", len(outliers))
```

```
mass Outliers: 9
mass_error_min Outliers: 11
mass_error_max Outliers: 11
mass_sini Outliers: 8
mass_sini_error_min Outliers: 14
mass_sini_error_max Outliers: 11
radius Outliers: 0
radius_error_min Outliers: 7
```

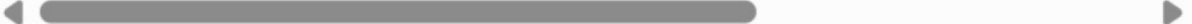
```
In [61]: df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

```
In [62]: df.fillna(df.median(numeric_only=True), inplace=True)
```

```
In [63]: df.describe()
```

Out[63]:

	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_error_min	m
count	112.000000	112.000000	112.000000	112.000000	112.000000	
mean	0.942266	0.142953	0.126312	0.962950	0.156575	
std	1.416519	0.483249	0.453695	1.395075	0.494969	
min	0.000910	0.000350	0.000000	0.000910	0.000148	
25%	0.036775	0.015548	0.012523	0.036843	0.017439	
50%	0.534500	0.050170	0.045000	0.572000	0.051000	
75%	1.171250	0.090250	0.090000	1.207500	0.091500	
max	8.890000	4.629000	4.629000	8.500000	4.628000	



In [64]: `df[numeric_columns].mean()`

Out[64]:

```

mass                0.942266
mass_error_min      0.142953
mass_error_max      0.126312
mass_sini            0.962950
mass_sini_error_min 0.156575
mass_sini_error_max 0.123264
radius              0.879408
radius_error_min    0.049080
dtype: float64

```

In [65]: `df[numeric_columns].median()`

Out[65]:

```

mass                0.53450
mass_error_min      0.05017
mass_error_max      0.04500
mass_sini            0.57200
mass_sini_error_min 0.05100
mass_sini_error_max 0.04000
radius              1.03500
radius_error_min    0.03015
dtype: float64

```

In [66]: `df[numeric_columns].mode().iloc[0]`

Out[66]:

```

mass                0.023
mass_error_min      0.130
mass_error_max      0.130
mass_sini            0.085
mass_sini_error_min 0.130
mass_sini_error_max 0.130
radius              1.039
radius_error_min    0.030
Name: 0, dtype: float64

```

In [67]:

```

range=df[numeric_columns].max() - df[numeric_columns].min()
print("Range: ",range)

```

```

Range: mass 8.889090
mass_error_min 4.628650
mass_error_max 4.629000
mass_sini 8.499090
mass_sini_error_min 4.627852
mass_sini_error_max 4.628000
radius 1.848000
radius_error_min 0.588000
dtype: float64

```

```
In [68]: df[numeric_columns].var()
```

```

Out[68]: mass 2.006527
mass_error_min 0.233530
mass_error_max 0.205839
mass_sini 1.946235
mass_sini_error_min 0.244994
mass_sini_error_max 0.203770
radius 0.276238
radius_error_min 0.004858
dtype: float64

```

```
In [69]: df[numeric_columns].std()
```

```

Out[69]: mass 1.416519
mass_error_min 0.483249
mass_error_max 0.453695
mass_sini 1.395075
mass_sini_error_min 0.494969
mass_sini_error_max 0.451409
radius 0.525584
radius_error_min 0.069699
dtype: float64

```

```

In [72]: df.to_csv(
    "cleaned_analysis_ready.csv",
    index=False
)

```

```
In [70]: df[numeric_columns].corr()
```

```

Out[70]:
```

	mass	mass_error_min	mass_error_max	mass_sini	mass_sini_err
mass	1.000000	0.493110	0.508340	0.986320	0.
mass_error_min	0.493110	1.000000	0.931543	0.491646	0.
mass_error_max	0.508340	0.931543	1.000000	0.507047	0.
mass_sini	0.986320	0.491646	0.507047	1.000000	0.
mass_sini_error_min	0.440628	0.961015	0.894876	0.476494	1.
mass_sini_error_max	0.464037	0.924609	0.991588	0.479546	0.
radius	0.392220	0.038886	0.004552	0.367776	0.
radius_error_min	0.158605	0.061689	0.064271	0.146257	0.

```
In [71]: df[numeric_columns].skew()
```

```
Out[71]: mass                2.979824  
mass_error_min            7.853242  
mass_error_max            9.084176  
mass_sini                  2.834550  
mass_sini_error_min       7.320372  
mass_sini_error_max       9.219767  
radius                   -0.124983  
radius_error_min          4.990595  
dtype: float64
```

Data Cleaning and Preprocessing Documentation

1. Missing Values

Missing values were checked using `isnull().sum()`. Numerical missing values were replaced with the median because the dataset contains skewed numerical variables. Median imputation is less affected by extreme values than mean imputation.

2. Duplicate Records

Duplicate records were checked using `duplicated().sum()`.

3. Data Types

The numerical columns were converted into appropriate numeric data types. Invalid nonnumeric values were converted into `NaN` so that they could be handled during the missing-value treatment.

4. Categorical Consistency

The categorical columns were checked for inconsistent formatting. Extra spaces and differences in capitalization were standardized. The `planet_status` column was found to contain only the category Confirmed

5. Invalid Values

Negative values were checked because measurements such as mass and radius should not be negative. Infinite values were also checked and replaced with missing values before performing statistical analysis.

6. Outliers

Outliers were detected using the Interquartile Range (IQR) method. A large number of outliers were found in several numerical columns. Since the dataset contains astronomical measurements, these extreme values may represent genuine observations. Therefore, the outliers were retained instead of being deleted.

7. Data Transformation

The numerical variables were examined for skewness. Log transformation can be applied to highly skewed variables such as mass, minimum mass and radius to reduce the effect of extreme values and improve the distribution.

8. Statistical Analysis

Descriptive statistics were calculated, including mean, median, minimum, maximum, range, variance and standard deviation. Correlation analysis was performed to understand relationships between numerical variables. Distribution analysis and statistical tests were also performed.

9. Final Validation

After cleaning, the final dataset was prepared for further data analysis, visualization and machine learning